

1.TWO EC2 SERVERS CREATING 2 DIFFERENT REGIONS IN TERRAFORM

Step 1: create an admin machine (amazon linux) and install the terraform on that with the help of terraform official site <https://terraform.io/> and get into the site check the installation links as per your request..

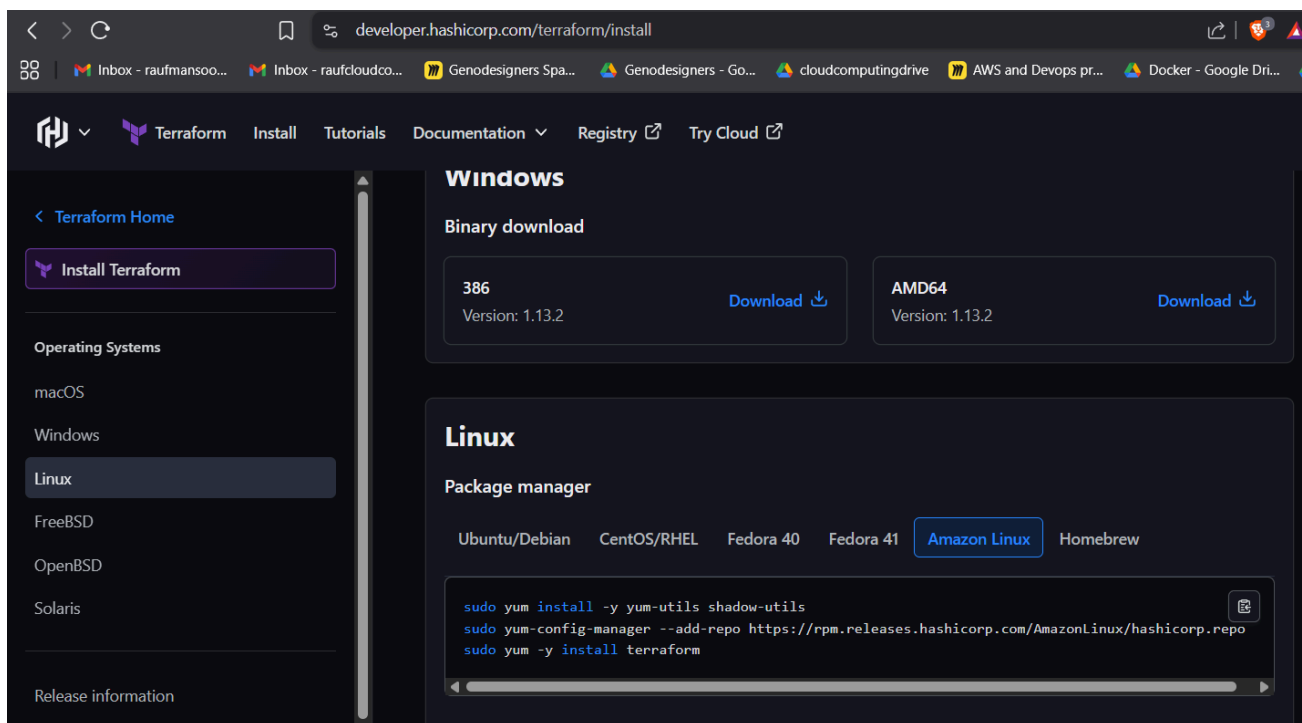
Here I used an amazon linux machine...
Installation link :

```
sudo yum install -y yum-utils shadow-utils
```

```
sudo yum-config-manager --add-repo
```

```
https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
```

```
sudo yum -y install terraform
```



Step 2: Once installed on server (admin) create a directory **mkdir ec2** and on that directory, create a files which are [provider.tf](#), [resource.tf](#), [variable.tf](#), [output.tf](#) using **touch command** and copy the scripts from VS code and paste into the files based on blocks.

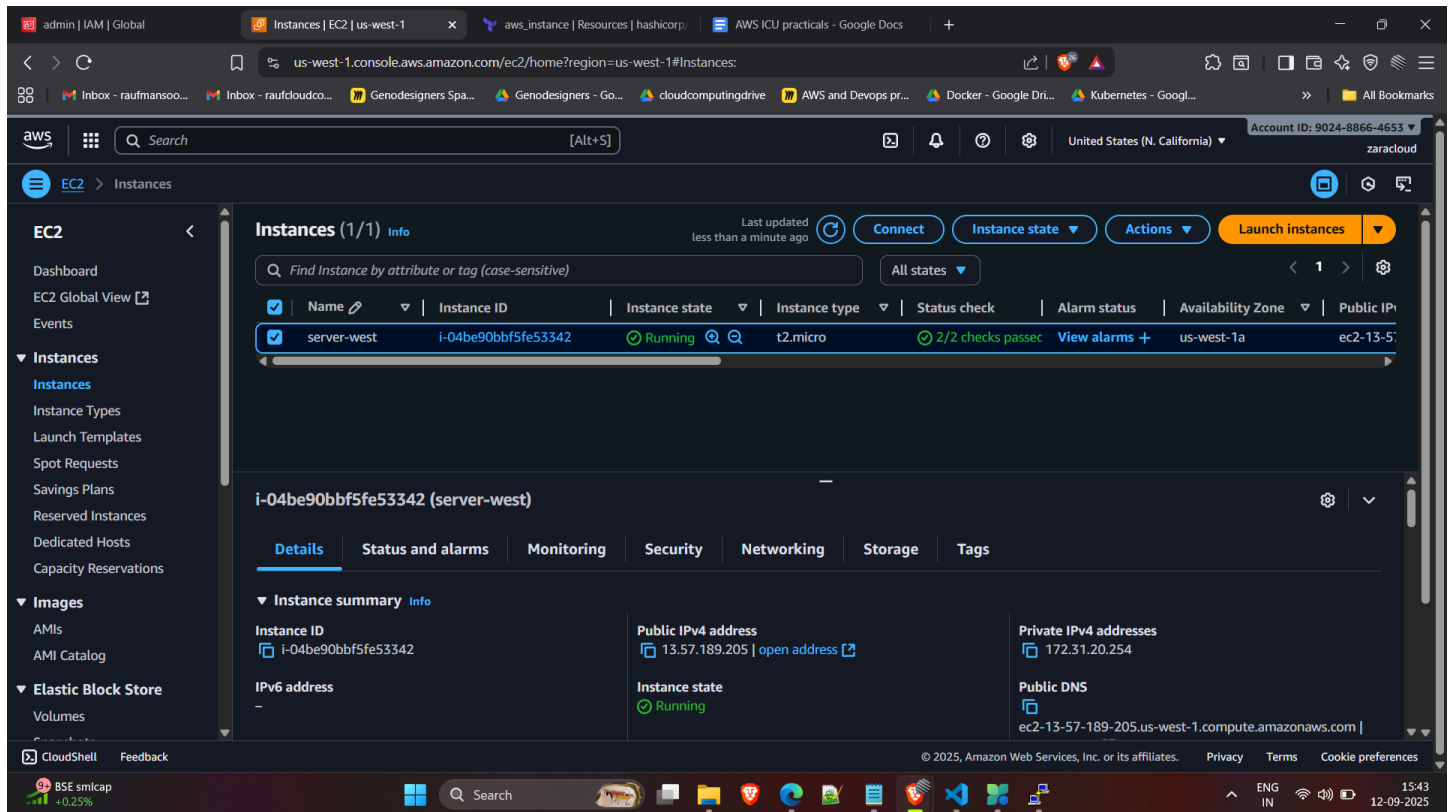
Note: Here we create resources in two different regions thats why we mentioned the alias name

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "us-east-1"  
  alias = "us_east"  
}  
  
provider "aws" {  
  region = "us-west-1"  
  alias = "us_west"}
```

```
}  
  
resource "aws_instance" "east_server" {  
  provider    = aws.us_east  
  ami         = "ami-0b09ffb6d8b58ca91" #amazonlinux  
  instance_type = "t2.micro"  
  
  tags = {  
    Name = "server-east"  
  }  
}
```

```
resource "aws_instance" "west_server" {  
  provider    = aws.us_west  
  ami         = "ami-0d9e15a8edf01ec21" #amazonlinux  
  instance_type = "t2.micro"  
  
  tags = {  
    Name = "server-west"  
  }  
}
```

Step 3: Once paste the codes in your files, Then run the command **terraform init > terraform validate > terraform plan > terraform apply** . Once all the processes done, it starts to create the resource in the backend which is AWS console.



To be continued.....

2. CREATING 3 SERVERS IN SAME REGION WITH DIFFERENT NAMES

Step 1: create an admin machine (amazon linux) and install the terraform on that with the help of terraform official site <https://terraform.io/> and get into the site check the installation links as per your request..

Here I used an amazon linux machine...

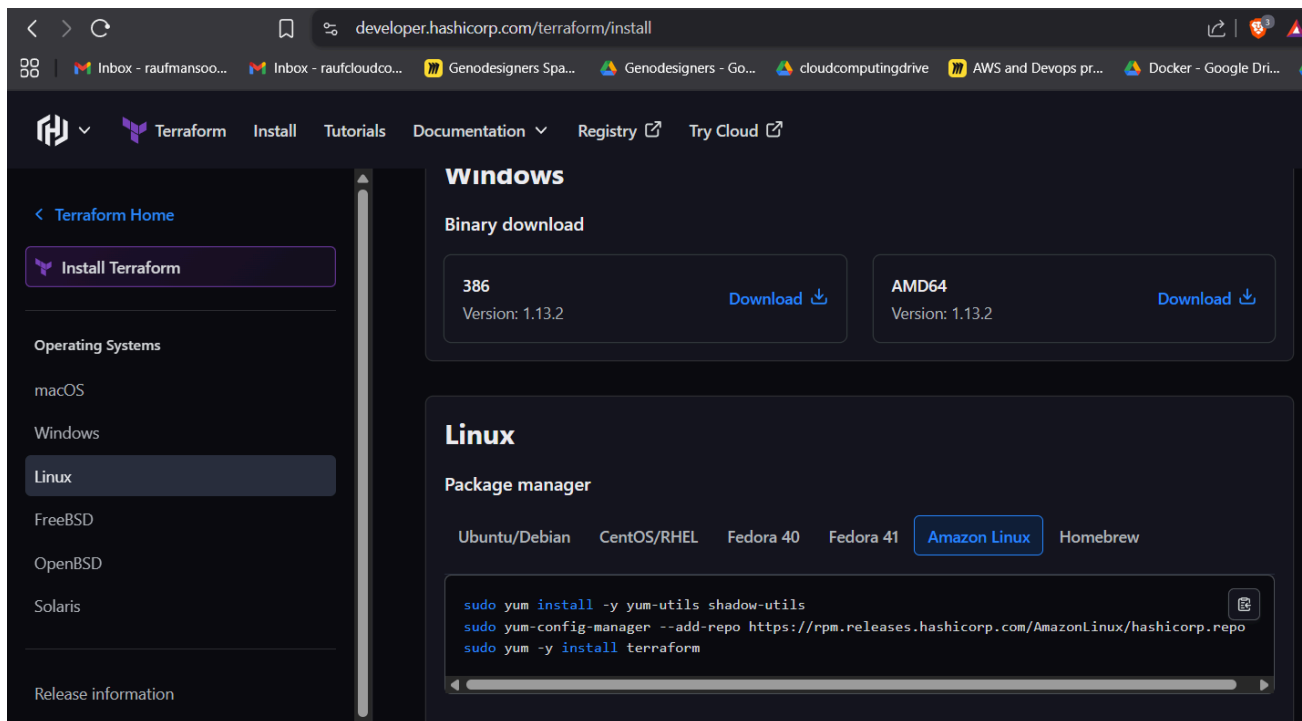
Installation link :

```
sudo yum install -y yum-utils shadow-utils
```

```
sudo yum-config-manager --add-repo
```

```
https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
```

```
sudo yum -y install terraform
```



Step 2: Once installed on server (admin) create a directory **mkdir ec2** and on that directory, create a files which are [provider.tf](#), [resource.tf](#), [variable.tf](#), [output.tf](#) using **touch command** and copy the scripts from VS code and paste into the files based on blocks.

Note: Here we use a loop operation for adding the dynamic names for our resources like.... `for_each = toset(["dev", "qa", "prod"])` and calling the loop function in tags.

Step 3: Once paste the codes in your files, Then run the command **terraform init > terraform validate > terraform plan > terraform apply** . Once all the processes are done, it starts to create the resource in the backend which is the AWS console.

```
terraform {  
  
  required_providers {  
  
    aws = {  
  
      source = "hashicorp/aws"  
  
      version = "~> 5.0"  
  
    }  
  
  }  
}  
  
provider "aws" {  
  
  region = "us-east-1"  
  
}  
  
resource "aws_instance" "servers" {  
  
  ami      = "ami-0b09ffb6d8b58ca91" #amazon linux  
  
  instance_type = "t2.micro"
```

```
for_each = toset(["dev", "qa", "prod"])
```

```
tags = {
```

```
  Name = each.key
```

```
  Env  = each.key
```

```
}
```

```
}
```

```
output "public_ips" {
```

```
  value = { for name, server in aws_instance.servers : name => server.public_ip }
```

```
}
```

```
output "private_ips" {
```

```
  value = { for name, server in aws_instance.servers : name => server.private_ip }
```

```
}
```



```
root@ip-172-31-22-0:~/ec2
[root@ip-172-31-22-0 ec2]# touch provider.tf
[root@ip-172-31-22-0 ec2]# touch resource.tf
[root@ip-172-31-22-0 ec2]# vi provider.tf
[root@ip-172-31-22-0 ec2]# vi resource.tf
[root@ip-172-31-22-0 ec2]# ls
provider.tf  resource.tf
[root@ip-172-31-22-0 ec2]# terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.100.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[root@ip-172-31-22-0 ec2]# terraform validate
Success! The configuration is valid.

[root@ip-172-31-22-0 ec2]# terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# aws_instance.servers["dev"] will be created
+ resource "aws_instance" "servers" {
  + ami           = "ami-0b09ffb6d8b58ca91"
  + arn           = (known after apply)
```

Check the below image which shows the public and private ip's while creating that's why we gave the output block in our script.

Once our resource created all the resource data will store in the state file which is the memory of terraform, without the [terraformstate.tf](#) file. It not able to track the resources.

```
root@ip-172-31-22-0:~# terraform apply
+ dev = (known after apply)
+ prod = (known after apply)
+ qa = (known after apply)
}

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

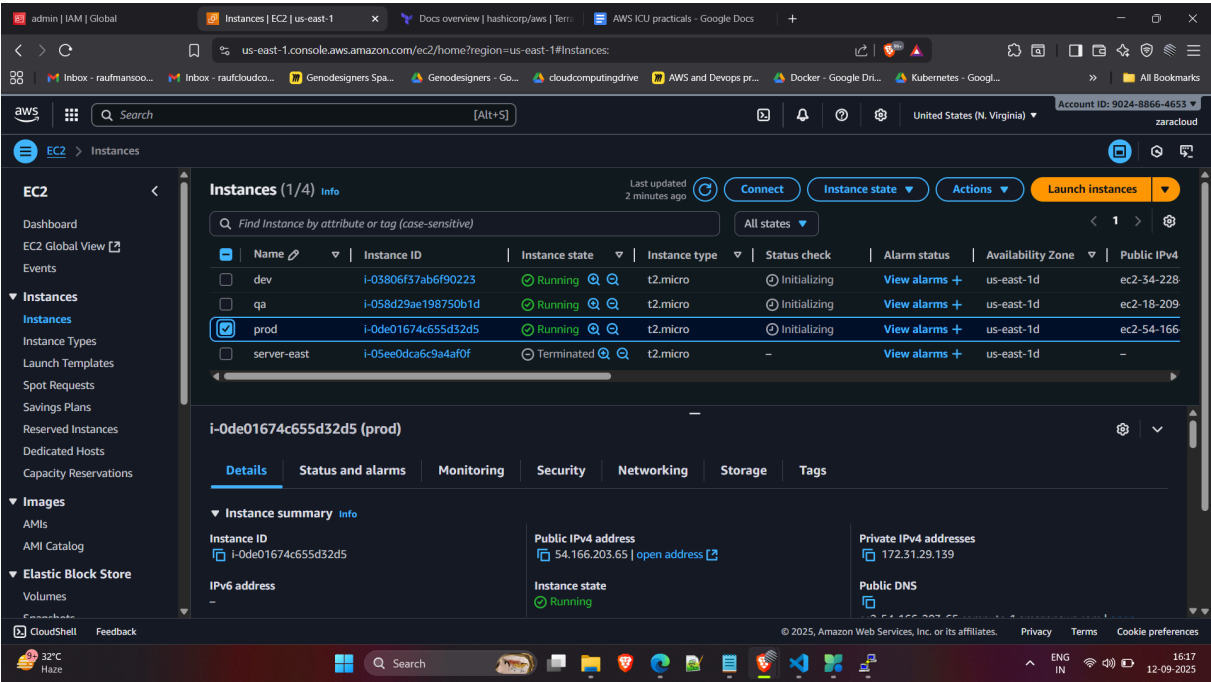
aws_instance.servers["prod"]: Creating...
aws_instance.servers["dev"]: Creating...
aws_instance.servers["qa"]: Creating...
aws_instance.servers["prod"]: Still creating... [00m10s elapsed]
aws_instance.servers["dev"]: Still creating... [00m10s elapsed]
aws_instance.servers["qa"]: Still creating... [00m10s elapsed]
aws_instance.servers["prod"]: Creation complete after 13s [id=i-00e93bef1e45c2898]
aws_instance.servers["dev"]: Creation complete after 12s [id=i-078093edb977733c6]
aws_instance.servers["qa"]: Creation complete after 13s [id=i-0dc3b68420baa92ba]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

private_ips = {
  "dev" = "172.31.24.217"
  "prod" = "172.31.31.29"
  "qa" = "172.31.21.197"
}
public_ips = {
  "dev" = "34.236.158.177"
  "prod" = "54.160.232.34"
  "qa" = "54.146.220.64"
}
[root@ip-172-31-22-0 ec2]#
```

Finally : once run all the terraform commands which starts to create the our resources in backend (aws console).



To be continued.....

3. CREATING THREE - S3 BUCKETS WITH NAMES LIKE 0,1,2,3 LIKE THAT.

Step 1: create an admin machine (amazon linux) and install the terraform on that with the help of terraform official site <https://terraform.io/> and get into the site check the installation links as per your request..

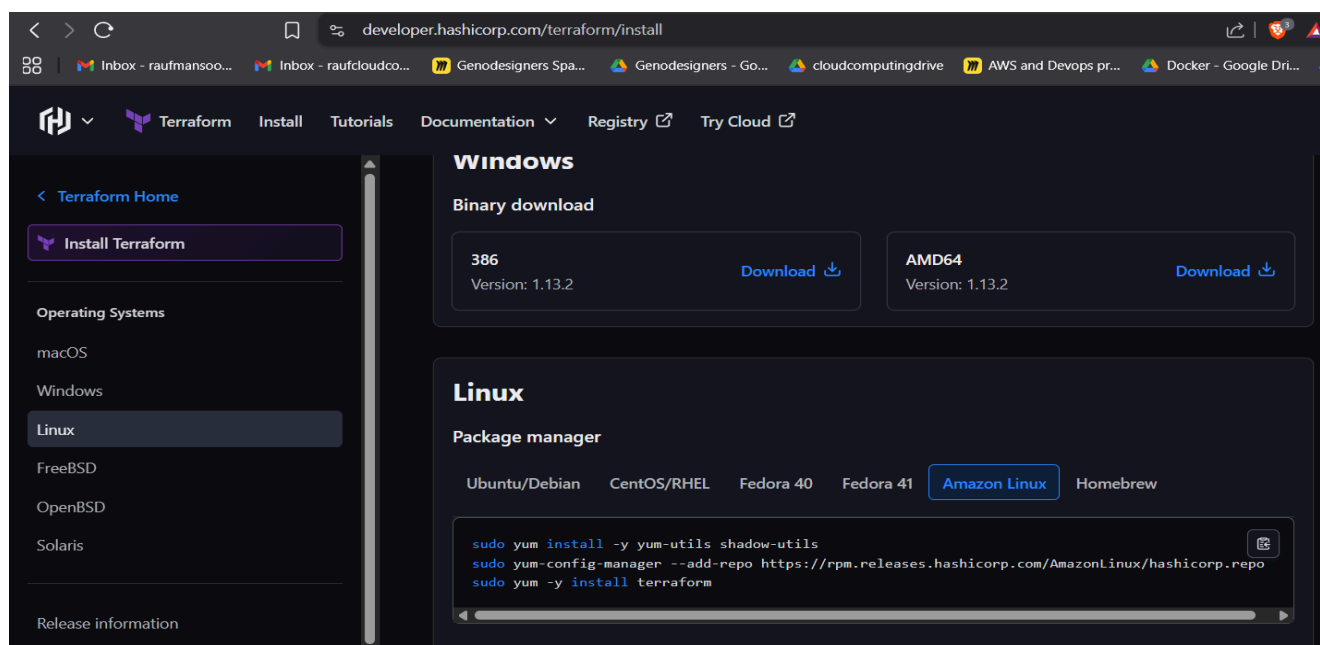
Here I used an amazon linux machine...
Installation link :

```
sudo yum install -y yum-utils shadow-utils
```

```
sudo yum-config-manager --add-repo
```

```
https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
```

```
sudo yum -y install terraform
```



Step 2: Once installed on server (admin) create a directory **mkdir ec2** and on that directory, create a files which are [provider.tf](#), [resource.tf](#), [variable.tf](#), [output.tf](#) using **touch command** and copy the scripts from VS code and paste into the files based on blocks.

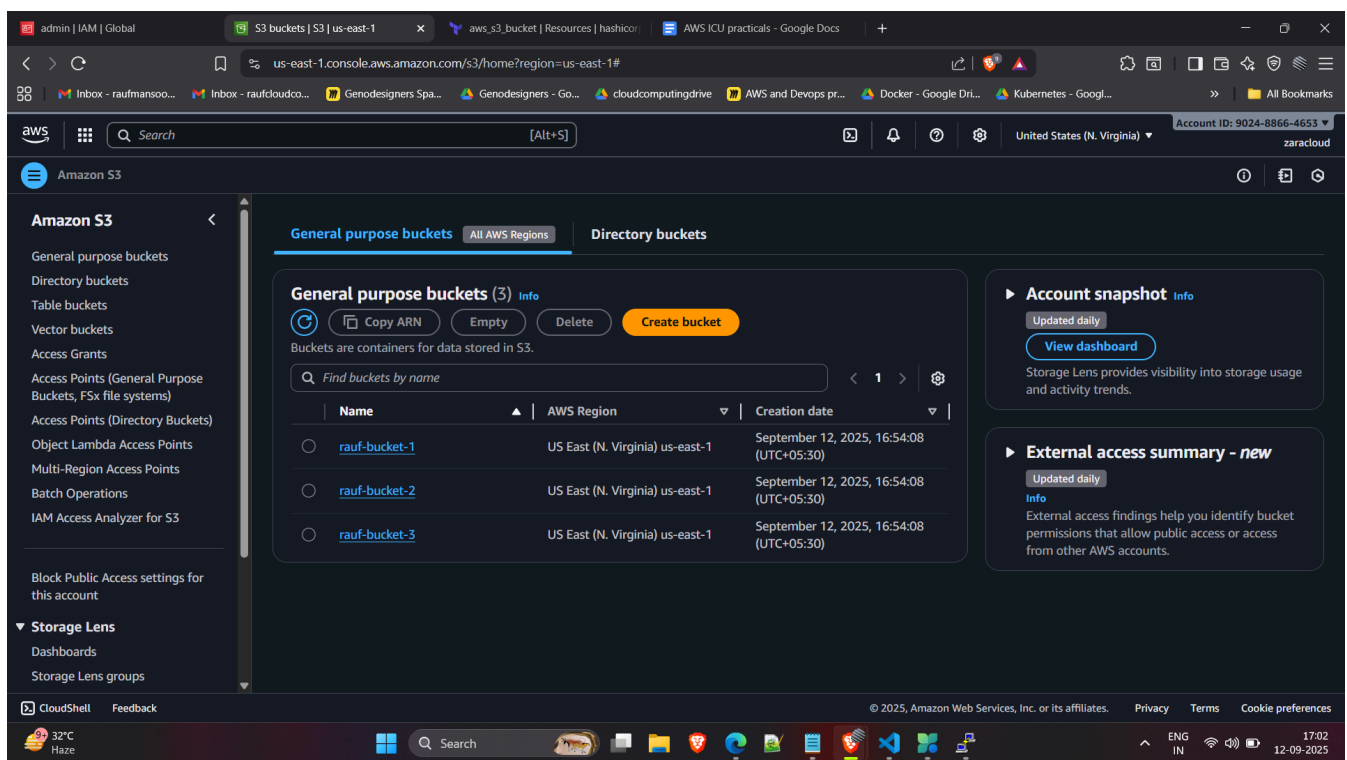
Note: Here we use count operation for the numerical name creation and the call that function with \$ symbol in count.index + 1

When we give count =3 operation it considered as aws_s3_buckets[0], aws_s3_buckets[1], aws_s3_buckets[2]. Then bucket name we give "rauf-bucket-\${count.index+1}" it gives name like **rauf-bucket-1 (0+1), rauf-bucket-2 (1+1), rauf-bucket-3 (2+1).**

code:

```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_s3_bucket" "buckets" {  
  count = 3  
  
  bucket = "rauf-bucket-${count.index + 1}"  
}
```

Step 3: Once paste the codes in your files, Then run the command **terraform init > terraform validate > terraform plan > terraform apply** . Once all the processes are done, it starts to create the resource in the backend which is the AWS console.



To be continued.....

4. Creating the 2 EC2 servers in 2 different regions with adding backend block and Terraform lifecycles

ADDING BACKEND BLOCK AND TERRAFORM LIFECYCEL WHICH IS
`create_before_destroy = true`

Step 1: create an admin machine (amazon linux) and install the terraform on that with the help of terraform official site <https://terraform.io/> and get into the site check the installation links as per your request..

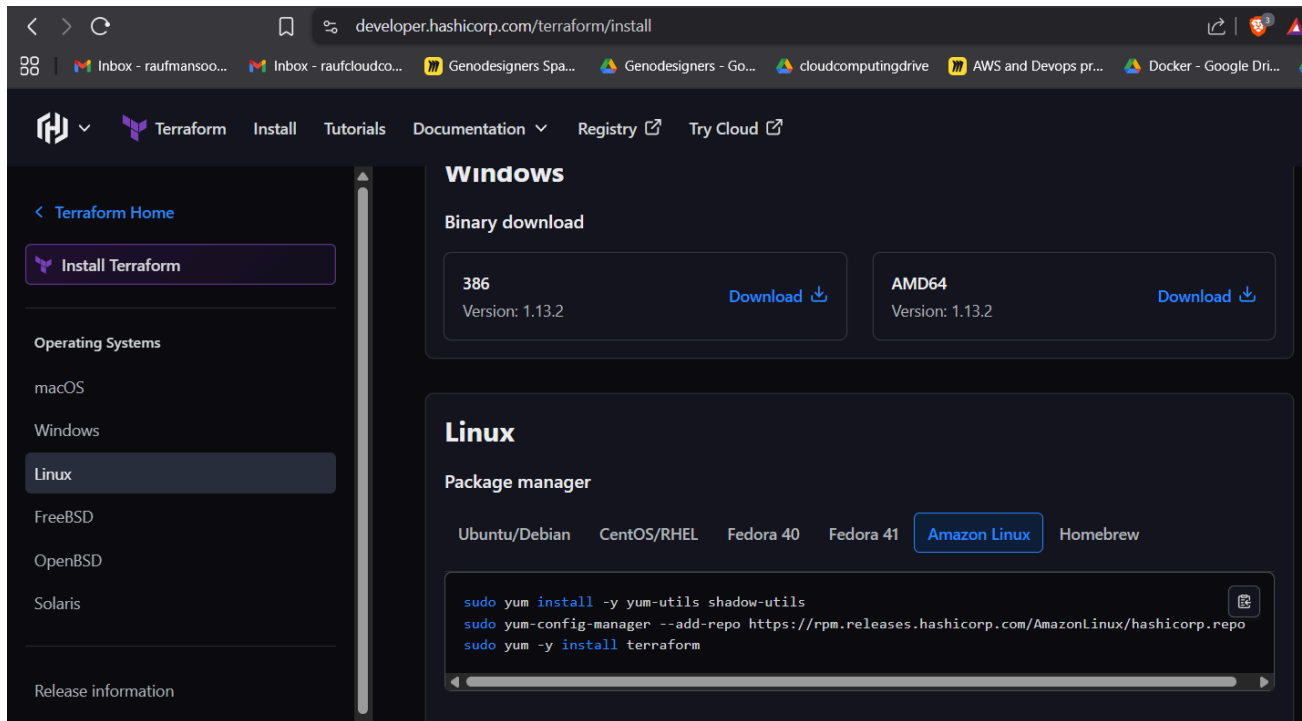
Here I used an amazon linux machine...
Installation link :

```
sudo yum install -y yum-utils shadow-utils
```

```
sudo yum-config-manager --add-repo
```

```
https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
```

```
sudo yum -y install terraform
```



Step 2: Once installed on server (admin) create a directory **mkdir ec2** and on that directory, create a files which are [provider.tf](#), [resource.tf](#), [variable.tf](#), [output.tf](#) using **touch command** and copy the scripts from VS code and paste into the files based on blocks.

Note : Here we add a backend block we mentioned our s3 bucket name and path. In this case whenever we run the terraform apply command it first checks the backend configuration and moves on to the installation process of providers plugin.... Based on this we can store our statefile securely in our remote bucket which is s3 or else it will store the state file in our local machine. It is not secure at all.

The `lifecycle { create_before_destroy = true }` only applies when Terraform needs to *replace* a resource (for example, you change the AMI, instance type, or tags).

```
resource "aws_instance" "east_server" {

  provider      = aws.us-east

  ami           = var.ami_id_east #amazonlinux

  instance_type = "t2.micro"

  tags = {

    Name = "server-east"

  }

  lifecycle {

    create_before_destroy = true

  }

}
```



```
}  
  
resource "aws_instance" "west_server" {  
  
    provider      = aws.us-west  
  
    ami           = var.ami_id_west #amazonlinux  
  
    instance_type = "t2.micro"  
  
    tags = {  
  
        Name = "server-west"  
  
    }  
  
    lifecycle {  
  
        create_before_destroy = true  
  
    }  
}
```

Note: We should add the lifecycle inside the resource block

Step 3: Once paste the codes in your files, Then run the command **terraform init > terraform validate > terraform plan > terraform apply** . Once all the processes are done, it starts to create the resource in the backend which is the AWS console.


```
root@ip-172-31-39-50:~/ec2

Changes to Outputs:
+ east_server_ips = {
+   private_ip = (known after apply)
+   public_ip  = (known after apply)
+ }
+ west_server_ips = {
+   private_ip = (known after apply)
+   public_ip  = (known after apply)
+ }

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

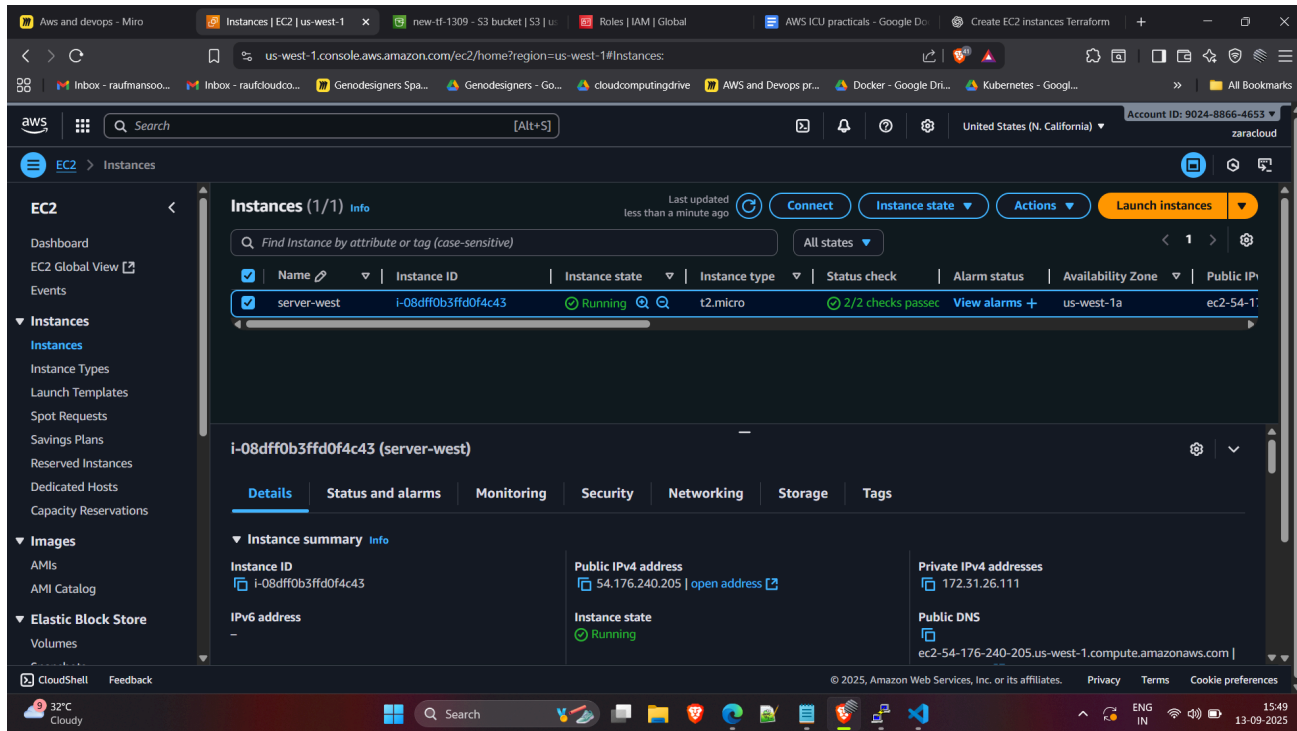
aws_instance.east_server: Creating...
aws_instance.west_server: Creating...
aws_instance.east_server: Still creating... [00m10s elapsed]
aws_instance.west_server: Still creating... [00m10s elapsed]
aws_instance.east_server: Creation complete after 13s [id=i-0c3c73d37af970517]
aws_instance.west_server: Creation complete after 16s [id=i-08dff0b3ffd0f4c43]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

east_server_ips = {
  "private_ip" = "172.31.20.223"
  "public_ip"  = "44.222.210.25"
}
west_server_ips = {
  "private_ip" = "172.31.26.111"
  "public_ip"  = "54.176.240.205"
}
[root@ip-172-31-39-50 ec2]#
```

The screenshot displays the AWS Management Console interface. The top navigation bar shows the account name 'zaracloud' and the region 'United States (N. Virginia)'. The left-hand navigation menu is expanded to 'EC2 > Instances'. The main content area, titled 'Instances (1/1) Info', contains a table with one instance listed: 'server-east' with Instance ID 'i-0c3c73d37af970517', state 'Running', type 't2.micro', and public IP 'ec2-44-222-210-25'. Below the table, the 'Details' tab for the selected instance is active, showing fields for Instance ID, Public IPv4 address (44.222.210.25), Private IPv4 addresses (172.31.20.223), and Public DNS (ec2-44-222-210-25.compute-1.amazonaws.com). The instance status is 'Running'.



DEPENDS ON :

```
resource "aws_instance" "east_server" {

  provider      = aws.us-east

  ami           = var.ami_id_east #amazonlinux
```

```
instance_type    = "t2.micro"
```

```
tags = {
```

```
    Name = "server-east"
```

```
}
```

```
lifecycle {
```

```
    create_before_destroy = true
```

```
}
```

```
}
```

```
resource "aws_instance" "west_server" {
```

```
    provider      = aws.us-west
```

```
    ami           = var.ami_id_west #amazonlinux
```

```
    instance_type = "t2.micro"
```

```
tags = {  
    Name = "server-west"  
}  
  
depends on = [aws_instance.east_server]  
}
```

Note : when i mentioned the depends on = [aws_instance.east_server] in resource block it will create the resource like depending on east server. For eg: once the east server is created then only the west server will create.

Please find the below before and after images for your understanding

Before:

```
root@ip-172-31-39-50:~/ec2
Changes to Outputs:
+ east_server_ips = {
+   private_ip = (known after apply)
+   public_ip  = (known after apply)
+ }
+ west_server_ips = {
+   private_ip = (known after apply)
+   public_ip  = (known after apply)
+ }

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.east_server: Creating...
aws_instance.west_server: Creating...
aws_instance.east_server: Still creating... [00m10s elapsed]
aws_instance.west_server: Still creating... [00m10s elapsed]
aws_instance.east_server: Creation complete after 13s [id=i-0c3c73d37af970517]
aws_instance.west_server: Creation complete after 16s [id=i-08dff0b3ffd0f4c43]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

east_server_ips = {
  "private_ip" = "172.31.20.223"
  "public_ip"  = "44.222.210.25"
}
west_server_ips = {
  "private_ip" = "172.31.26.111"
  "public_ip"  = "54.176.240.205"
}
[root@ip-172-31-39-50 ec2]#
```

After:

```
root@ip-172-31-39-50:~/ec2
Changes to Outputs:
+ east_server_ips = {
+   private_ip = (known after apply)
+   public_ip  = (known after apply)
+ }
+ west_server_ips = {
+   private_ip = (known after apply)
+   public_ip  = (known after apply)
+ }

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.east_server: Creating...
aws_instance.east_server: Still creating... [00m10s elapsed]
aws_instance.east_server: Creation complete after 13s [id=i-0ce02a6820a8be006]
aws_instance.west_server: Creating...
aws_instance.west_server: Still creating... [00m10s elapsed]
aws_instance.west_server: Creation complete after 13s [id=i-038f87df3c7da5894]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

east_server_ips = {
  "private_ip" = "172.31.21.25"
  "public_ip"  = "18.212.226.174"
}
west_server_ips = {
  "private_ip" = "172.31.19.112"
  "public_ip"  = "13.56.197.49"
}
[root@ip-172-31-39-50 ec2]#
```


PREVENT DESTROY LIFECYCLE

```
resource "aws_instance" "east_server" {

    provider      = aws.us-east

    ami           = var.ami_id_east #amazonlinux

    instance_type = "t2.micro"

    tags = {

        Name = "server-east"

    }

    lifecycle {

        prevent_destroy = true

    }

}

resource "aws_instance" "west_server" {
```

```
provider      = aws.us-west

ami           = var.ami_id_west #amazonlinux

instance_type = "t2.micro"

tags = {

    Name = "server-west"

}

depends_on = [aws_instance.east_server]

lifecycle {

    prevent_destroy = true

}
```

Result : when try to destroy it showing then error

```
root@ip-172-31-39-50:~# terraform destroy -auto-approve

- delete_on_termination = true -> null
- device_name            = "/dev/xvda" -> null
- encrypted              = false -> null
- iops                   = 3000 -> null
- tags                   = {} -> null
- tags_all               = {} -> null
- throughput             = 125 -> null
- volume_id              = "vol-02dd5c847cd5ee478" -> null
- volume_size            = 8 -> null
- volume_type            = "gp3" -> null
# (1 unchanged attribute hidden)

}

Plan: 0 to add, 0 to change, 1 to destroy.

Changes to Outputs:
- east_server_ips = {
  - private_ip = "172.31.22.214"
  - public_ip  = "13.220.176.40"
} -> null
- west_server_ips = {
  - private_ip = "172.31.27.241"
  - public_ip  = "54.193.24.178"
} -> null

Error: Instance cannot be destroyed

on resource.tf line 15:
15: resource "aws_instance" "west_server" {

Resource aws_instance.west_server has lifecycle.prevent_destroy set, but the plan calls for this resource to be destroyed. To avoid this error and continue with the plan, either disable lifecycle.prevent_destroy or reduce the scope of the plan using the -target option.

[root@ip-172-31-39-50 ec2]#
```

Then how to delete the resources go to resource file change the lifecycle prevent_destroy = false

Now again run the terraform destroy command it will delete.

Do you really want to destroy all resources?

Terraform will destroy all your managed infrastructure, as shown above.

There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

```
aws_instance.west_server: Destroying... [id=i-042ce0cc7ab3ed63d]
aws_instance.west_server: Still destroying... [id=i-042ce0cc7ab3ed63d, 00m10s elapsed]
aws_instance.west_server: Still destroying... [id=i-042ce0cc7ab3ed63d, 00m20s elapsed]
aws_instance.west_server: Still destroying... [id=i-042ce0cc7ab3ed63d, 00m30s elapsed]
aws_instance.west_server: Destruction complete after 31s
aws_instance.east_server: Destroying... [id=i-0aa5a45dbff2551d8]
aws_instance.east_server: Still destroying... [id=i-0aa5a45dbff2551d8, 00m10s elapsed]
aws_instance.east_server: Still destroying... [id=i-0aa5a45dbff2551d8, 00m20s elapsed]
aws_instance.east_server: Still destroying... [id=i-0aa5a45dbff2551d8, 00m30s elapsed]
aws_instance.east_server: Still destroying... [id=i-0aa5a45dbff2551d8, 00m40s elapsed]
aws_instance.east_server: Still destroying... [id=i-0aa5a45dbff2551d8, 00m50s elapsed]
aws_instance.east_server: Still destroying... [id=i-0aa5a45dbff2551d8, 01m00s elapsed]
aws_instance.east_server: Still destroying... [id=i-0aa5a45dbff2551d8, 01m10s elapsed]
aws_instance.east_server: Still destroying... [id=i-0aa5a45dbff2551d8, 01m20s elapsed]
aws_instance.east_server: Destruction complete after 1m21s
```

Destroy complete! Resources: 2 destroyed.

[root@ip-172-31-39-50 ec2]#

Ignore_changes Lifecycle

```
resource "aws_instance" "east_server" {  
  
    provider      = aws.us-east  
  
    ami           = var.ami_id_east #amazonlinux  
  
    instance_type = "t2.micro"  
  
  
    tags = {  
  
        Name = "server-east"  
  
    }  
  
  
    lifecycle {  
  
        ignore_changes = [tags]  
  
    }  
}
```

```
resource "aws_instance" "west_server" {

  provider      = aws.us-west

  ami           = var.ami_id_west #amazonlinux

  instance_type = "t2.micro"

  tags = {

    Name = "server-west"

  }

  depends_on = [aws_instance.east_server]

  lifecycle {

    ignore_chnages = [tags]

  }

}
```

What happens

- If you (or someone) updates the EC2 tags directly in AWS (console/CLI),
Terraform will not try to revert them on the next terraform apply.
- But if you change the AMI or instance_type in Terraform, it will still manage those normally.

Even if i have done any changes in the console it will ignore that i create the resource as per existing updated state file.

```
root@ip-172-31-39-50:~/ec2
Changes to Outputs:
+ east_server_ips = {
+   private_ip = (known after apply)
+   public_ip  = (known after apply)
+ }
+ west_server_ips = {
+   private_ip = (known after apply)
+   public_ip  = (known after apply)
+ }

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.east_server: Creating...
aws_instance.east_server: Still creating... [00m10s elapsed]
aws_instance.east_server: Creation complete after 12s [id=i-0c9c8d60ec4e1ad48]
aws_instance.west_server: Creating...
aws_instance.west_server: Still creating... [00m10s elapsed]
aws_instance.west_server: Creation complete after 13s [id=i-05f48ffb2044013c3]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

east_server_ips = {
  "private_ip" = "172.31.28.193"
  "public_ip"  = "3.208.27.59"
}
west_server_ips = {
  "private_ip" = "172.31.26.83"
  "public_ip"  = "13.56.149.124"
}
[root@ip-172-31-39-50 ec2]#
```

THANKS YOU